

Ascend Extension for PyTorch 7.0.0

软件安装指南

文档版本 01
发布日期 2025-10-15



版权所有 © 华为技术有限公司 2025。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

安全声明

产品生命周期政策

华为公司对产品生命周期的规定以“产品生命周期终止政策”为准，该政策的详细内容请参见如下网址：
<https://support.huawei.com/ecolumnsweb/zh/warranty-policy>

漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
<https://www.huawei.com/cn/psirt/vul-response-process>
如企业客户须获取漏洞信息，请参见如下网址：
<https://securitybulletin.huawei.com/enterprise/cn/security-advisory>

华为初始证书权责说明

华为公司对随设备出厂的初始数字证书，发布了“华为设备初始数字证书权责说明”，该说明的详细内容请参见如下网址：
<https://support.huawei.com/enterprise/zh/bulletins-service/ENEWS2000015766>

华为企业业务最终用户许可协议(EULA)

本最终用户许可协议是最终用户（个人、公司或其他任何实体）与华为公司就华为软件的使用所缔结的协议。最终用户对华为软件的使用受本协议约束，该协议的详细内容请参见如下网址：
<https://e.huawei.com/cn/about/eula>

产品资料生命周期策略

华为公司针对随产品版本发布的售后客户资料（产品资料），发布了“产品资料生命周期策略”，该策略的详细内容请参见如下网址：
<https://support.huawei.com/enterprise/zh/bulletins-website/ENEWS2000017760>

目录

- 1 安装说明..... 1
- 2 安装前准备..... 3
- 3 安装 PyTorch..... 5
 - 3.1 方式一：二进制软件包安装..... 5
 - 3.2 方式二：源码编译安装..... 16
 - 3.3 方式三：二进制软件包安装（abi1 版本）..... 18
- 4（可选）安装扩展模块..... 20
 - 4.1 编译 libtorch_npu..... 20
 - 4.2 安装 APEX 模块..... 22
 - 4.3 安装 torchvision..... 24
- 5 卸载..... 25
- 6 升级..... 27
- 7 参考信息..... 28
 - 7.1 配置 pip 源..... 28
 - 7.2 安装 10.2.0 版本 gcc..... 28
 - 7.3 安装 3.18.0 版本 cmake..... 30

1 安装说明

为使用PyTorch框架的开发者提供昇腾AI处理器的超强算力，昇腾开发Ascend Extension for PyTorch（即torch_npu插件）用于适配PyTorch框架。

本文主要向用户介绍如何快速完成PyTorch框架、Ascend Extension for PyTorch（即torch_npu插件）以及扩展模块的安装。

安装方案

本文档包含物理机、容器、虚拟机场景下，安装驱动、固件、CANN软件、PyTorch框架和torch_npu插件的方案，部署架构如图1-1所示。

图 1-1 安装方案



硬件配套和支持的操作系统

表 1-1 产品硬件支持列表

产品	是否支持（训练场景）
Atlas A3 训练系列产品	√

产品	是否支持（训练场景）
Atlas A3 推理系列产品	x
Atlas A2 训练系列产品	√
Atlas 800I A2 推理产品	x
Atlas 200I/500 A2 推理产品	x
Atlas 推理系列产品	x
Atlas 训练系列产品	√

- 各硬件产品对应物理机部署场景支持的操作系统请参考[兼容性查询助手](#)。
- 各硬件产品对应虚拟机部署场景支持的操作系统请参考《CANN软件安装指南》的“[安装说明](#)”章节。
- 各硬件产品对应容器部署场景支持的操作系统请参考《CANN软件安装指南》的“[安装说明](#)”章节。

安装方式

本手册提供了二进制软件包、源码编译以及二进制软件包（abi1版本）安装方式，用户可根据实际需求自行选择安装PyTorch框架和torch_npu插件的方式，不要求两者安装方式统一。

2 安装前准备

若用户使用预先训练好的模型进行数据处理和分析，即仅进行离线推理，请跳过此章节。

若用户进行训练或者在线推理，请参考以下完成安装前准备。

- 安装配套版本的NPU驱动固件、CANN软件（Toolkit、Kernels和NNAL）并配置CANN环境变量，具体请参见《[CANN 软件安装指南](#)》。

CANN软件提供进程级环境变量设置脚本，训练或推理场景下使用NPU执行业务代码前需要调用该脚本，否则业务代码将无法执行。

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

以上命令以Toolkit包为例，其他CANN软件请用户根据set_env.sh的实际路径执行命令。

- 容器场景下源码编译安装torch_npu插件，涉及从外部网络获取社区提供基础镜像、Python三方库以及编译使用源码，代理配置等相关网络问题请参考[Docker官方文档](#)。
- 根据不同类型的操作系统安装所需依赖前，请在安装用户下检查源是否可用。以配置华为镜像源为例，可参考[华为开源镜像站](#)中镜像源对应的配置方法操作。
- Ascend Extension for PyTorch支持安装abi1版本的安装包，具体请参见[3.3 方式三：二进制软件包安装（abi1版本）](#)。

从GCC 5.1版本开始，libstdc++为了更好的实现C++11规范，更改了std::string和std::list的一些接口，导致新老版本ABI不兼容。在C++代码编译时使用-D_GLIBCXX_USE_CXX11_ABI宏选择新老版本ABI。

- 当安装abi1版本时，使用新版本ABI。
- 当安装abi0版本时，使用老版本ABI。

用户可根据实际情况的ABI版本，选择合适ABI的安装包。

- 通过源码编译安装PyTorch框架和torch_npu插件时，需安装如下环境依赖。

```
pip3 install pyyaml  
pip3 install wheel  
pip3 install setuptools
```

如果使用非root用户安装，需要在命令后加--user，例如：**pip3 install pyyaml --user**。

须知

建议使用非root用户安装运行torch_npu，且建议对安装程序的目录文件做好权限管控：文件夹权限设置为750，文件权限设置为640。可以通过设置umask控制安装后文件的权限，如设置umask为0027。

更多安全相关内容请参见《[安全声明](#)》中各组件关于“文件权限控制”的说明。

3 安装 PyTorch

- 方式一：二进制软件包安装
- 方式二：源码编译安装
- 方式三：二进制软件包安装（abi1版本）

3.1 方式一：二进制软件包安装

通过wheel格式的二进制软件包直接安装PyTorch框架和torch_npu插件。

执行安装命令前，请参见2 安装前准备完成环境变量配置及其他环境准备。

安装 PyTorch 框架

表 3-1 PyTorch 框架安装

PyTorch 版本	Python 版本	系统架构	CANN 版本	安装方式	安装命令
2.1.0	Python 3.8	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.1.0-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.8	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0%2Bcpu-cp38-cp38-linux_x86_64.whl # 安装命令 pip3 install torch-2.1.0+cpu-cp38-cp38-linux_x86_64.whl
	Python 3.9	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.1.0-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch 版本	Python 版本	系统架构	CANN 版本	安装方式	安装命令
	Python 3.9	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0%2Bcpu-cp39-cp39-linux_x86_64.whl # 安装命令 pip3 install torch-2.1.0+cpu-cp39-cp39-linux_x86_64.whl
	Python 3.10	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.1.0-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.10	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0%2Bcpu-cp310-cp310-linux_x86_64.whl # 安装命令 pip3 install torch-2.1.0+cpu-cp310-cp310-linux_x86_64.whl
	Python 3.11	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.1.0-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.11	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.1.0%2Bcpu-cp311-cp311-linux_x86_64.whl # 安装命令 pip3 install torch-2.1.0+cpu-cp311-cp311-linux_x86_64.whl
2.3.1	Python 3.8	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.3.1-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.8	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1%2Bcpu-cp38-cp38-linux_x86_64.whl # 安装命令 pip3 install torch-2.3.1+cpu-cp38-cp38-linux_x86_64.whl
	Python 3.9	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.3.1-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch 版本	Python 版本	系统架构	CANN 版本	安装方式	安装命令
	Python 3.9	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1%2Bcpu-cp39-cp39-linux_x86_64.whl # 安装命令 pip3 install torch-2.3.1+cpu-cp39-cp39-linux_x86_64.whl
	Python 3.10	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.3.1-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.10	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1%2Bcpu-cp310-cp310-linux_x86_64.whl # 安装命令 pip3 install torch-2.3.1+cpu-cp310-cp310-linux_x86_64.whl
	Python 3.11	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.3.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.11	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.3.1%2Bcpu-cp311-cp311-linux_x86_64.whl # 安装命令 pip3 install torch-2.3.1+cpu-cp311-cp311-linux_x86_64.whl
2.4.0	Python 3.8	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.4.0-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.8	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0%2Bcpu-cp38-cp38-linux_x86_64.whl # 安装命令 pip3 install torch-2.4.0+cpu-cp38-cp38-linux_x86_64.whl
	Python 3.9	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.4.0-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch 版本	Python 版本	系统架构	CANN 版本	安装方式	安装命令
	Python 3.9	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0%2Bcpu-cp39-cp39-linux_x86_64.whl # 安装命令 pip3 install torch-2.4.0+cpu-cp39-cp39-linux_x86_64.whl
	Python 3.10	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.4.0-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.10	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0%2Bcpu-cp310-cp310-linux_x86_64.whl # 安装命令 pip3 install torch-2.4.0+cpu-cp310-cp310-linux_x86_64.whl
	Python 3.11	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.4.0-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.11	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.4.0%2Bcpu-cp311-cp311-linux_x86_64.whl # 安装命令 pip3 install torch-2.4.0+cpu-cp311-cp311-linux_x86_64.whl
2.5.1	Python 3.9	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.5.1-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.5.1-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.9	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.5.1%2Bcpu-cp39-cp39-linux_x86_64.whl # 安装命令 pip3 install torch-2.5.1+cpu-cp39-cp39-linux_x86_64.whl
	Python 3.10	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.5.1-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.5.1-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch版本	Python版本	系统架构	CANN版本	安装方式	安装命令
	Python 3.10	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.5.1%2Bcpu-cp310-cp310-linux_x86_64.whl # 安装命令 pip3 install torch-2.5.1+cpu-cp310-cp310-linux_x86_64.whl
	Python 3.11	AArch64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.5.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch-2.5.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	Python 3.11	X86_64	8.1.RC1	Pip	# 下载软件包 wget https://download.pytorch.org/whl/cpu/torch-2.5.1%2Bcpu-cp311-cp311-linux_x86_64.whl # 安装命令 pip3 install torch-2.5.1+cpu-cp311-cp311-linux_x86_64.whl

更多PyTorch版本可单击[Link](#)进行查询。

安装 torch_npu 插件

表 3-2 安装 torch_npu 插件

PyTorch版本	torch_npu插件版本	Python版本	系统架构	CANN版本	安装方式	安装命令
2.1.0	7.0.0	Python 3.8	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch版本	torch_npu插件版本	Python版本	系统架构	CANN版本	安装方式	安装命令
	7.0.0	Python 3.8	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.9	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.9	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.10	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.10	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl

PyTorch版本	torch_npu插件版本	Python版本	系统架构	CANN版本	安装方式	安装命令
	7.0.0	Python 3.11	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.11	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.1.0.post12-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
2.3.1	7.0.0	Python 3.8	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.8	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.9	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch 版本	torch_npu 插件版本	Python 版本	系统架构	CANN 版本	安装方式	安装命令
	7.0.0	Python 3.9	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.10	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.10	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.11	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.11	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.3.1/torch_npu-2.3.1.post6-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.3.1.post6-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl

PyTorch版本	torch_npu插件版本	Python版本	系统架构	CANN版本	安装方式	安装命令
2.4.0	7.0.0	Python 3.8	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp38-cp38-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.8	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.9	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.9	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.10	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl

PyTorch 版本	torch_npu 插件版本	Python 版本	系统架构	CANN 版本	安装方式	安装命令
	7.0.0	Python 3.10	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.11	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.11	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.4.0/torch_npu-2.4.0.post4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.4.0.post4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
2.5.1	7.0.0	Python 3.9	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.5.1/torch_npu-2.5.1-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.5.1-cp39-cp39-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.9	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.5.1/torch_npu-2.5.1-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.5.1-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl

PyTorch版本	torch_npu插件版本	Python版本	系统架构	CANN版本	安装方式	安装命令
	7.0.0	Python 3.10	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.5.1/torch_npu-2.5.1-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.5.1-cp310-cp310-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.10	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.5.1/torch_npu-2.5.1-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.5.1-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
	7.0.0	Python 3.11	AArch64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.5.1/torch_npu-2.5.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl # 安装命令 pip3 install torch_npu-2.5.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
	7.0.0	Python 3.11	X86_64	8.1.RC1	Pip	# 下载插件包 wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.5.1/torch_npu-2.5.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl # 安装命令 pip3 install torch_npu-2.5.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl

更多PyTorch插件版本可单击[Link](#)查询。

 说明

若PyTorch版本为2.1.0，出现“找不到google或者protobuf或者protobuf版本过高”报错时，需执行如下命令：

```
pip3 install protobuf==3.20
```

验证是否成功安装

- 方法一

```
python3 -c "import torch;import torch_npu; a = torch.randn(3, 4).npu(); print(a + a);"
```

输出如下类似信息说明PyTorch框架与插件安装成功。

```
tensor([[[-0.6066, 6.3385, 0.0379, 3.3356],  
        [ 2.9243, 3.3134, -1.5465, 0.1916],  
        [-2.1807, 0.2008, -1.1431, 2.1523]], device='npu:0')
```

- 方法二

```
import torch  
import torch_npu
```

```
x = torch.randn(2, 2).npu()  
y = torch.randn(2, 2).npu()  
z = x.mm(y)
```

```
print(z)
```

输出如下类似信息说明PyTorch框架与插件安装成功。

```
tensor([[[-0.0515, 0.3664],  
        [-0.1258, -0.5425]], device='npu:0')
```

3.2 方式二：源码编译安装

编译安装适用于二次开发场景，如自定义算子适配开发后，用户可以选择需要的分支版本自行编译PyTorch框架和torch_npu插件。

执行安装命令前，请参见[2 安装前准备](#)完成环境变量配置及其他环境准备。

安装 PyTorch 框架

具体步骤请参见[PyTorch官网](#)。

安装 torch_npu 插件

以下操作步骤以安装PyTorch 2.1.0版本为例。

- 方式一（推荐）：容器场景

- a. 下载torch_npu源码。

```
git clone https://gitee.com/ascend/pytorch.git -b v2.1.0-7.0.0 --depth 1
```

以v2.1.0-7.0.0为例，下载对应的Ascend Extension for PyTorch代码分支。请参见《版本说明》中的“[相关产品版本配套说明](#)”章节下载Ascend Extension for PyTorch其他版本的分支代码。

- b. 构建镜像。

```
cd pytorch/ci/docker/{arch}  
docker build -t manylinux-builder:v1 .
```

说明

- `{arch}`表示CPU架构（X86或ARM）。
 - 注意不要遗漏命令结尾的“.”。
 - c. 进入Docker容器，并将torch_npu源代码挂载至容器内。

```
docker run -it -v {code_path}/pytorch:/home/pytorch manylinux-builder:v1 bash
```


`{code_path}`表示torch_npu源代码路径，请根据实际情况进行替换。
 - d. 编译生成二进制安装包。

```
cd /home/pytorch
bash ci/build.sh --python=3.8
```

如需指定其他Python版本请使用--python=3.9、--python=3.10或--python=3.11。

- e. 在运行环境中安装生成的插件torch_npu包，如果使用非root用户安装，需要在命令后加--user。

```
pip3 install --upgrade dist/torch_npu-2.1.0.post12-cp38-cp38-linux_aarch64.whl
```

请用户根据实际情况更改命令中的torch_npu包名。

- 方式二：物理机及虚拟机场景

- a. 安装依赖。

选择编译安装方式安装时需要安装系统依赖，根据不同类型的操作系统，选择对应的命令安装所需依赖。

- openEuler系列（openEuler、CentOS、Kylin、BCLinux、BC-Linux-for-Euler、UOS20 1050e、UOS20 1020e、UOSV20、AntOS、CTyunOS、CULinux、Tlinux）：

- 1) 安装依赖（除gcc和cmake以外）。

```
yum install -y patch libjpeg-turbo-devel dos2unix openblas git
```

- 2) 安装gcc和cmake。

根据实际情况，安装对应gcc和cmake版本，版本信息及安装指导请参见以下说明。

- Debian系列（Ubuntu、Debian、UOS20、UOS20 SP1）：

- 1) 安装依赖（除gcc和cmake以外）。

```
apt-get install -y patch build-essential libbz2-dev libreadline-dev wget curl llvm
libncurses5-dev libncursesw5-dev xz-utils tk-dev liblzma-dev m4 dos2unix
libopenblas-dev git
```

- 2) 安装gcc和cmake。

根据实际情况，安装对应gcc和cmake版本，版本信息及安装指导请参见以下说明。

说明

AArch64架构安装gcc10.2.0版本，X86_64架构安装gcc9.4.0版本，cmake则需为3.18.0版本及以上，安装指导可参见[7.2 安装10.2.0版本gcc](#)和[7.3 安装3.18.0版本cmake](#)。

- b. 编译生成torch_npu插件的二进制安装包。

- i. 以v2.1.0-7.0.0为例，下载对应的Ascend Extension for PyTorch分支代码并进入插件根目录。

```
git clone -b v2.1.0-7.0.0 https://gitee.com/ascend/pytorch.git
cd pytorch
```

请参见《版本说明》中的“[相关产品版本配套说明](#)”章节下载Ascend Extension for PyTorch其他版本的分支代码。

- ii. 编译生成二进制安装包。

```
bash ci/build.sh --python=3.8
```

如需指定其他Python版本请使用--python=3.9、--python=3.10或--python=3.11。

- c. 安装pytorch/dist目录下生成的插件torch_npu包，如果使用非root用户安装，需要在命令后加--user。

```
pip3 install --upgrade dist/torch_npu-2.1.0.post12-cp38-cp38-linux_aarch64.whl
```

请用户根据实际情况更改命令中的torch_npu包名。

验证是否成功安装

- 方法一

```
python3 -c "import torch;import torch_npu; a = torch.randn(3, 4).npu(); print(a + a);"
```

输出如下类似信息说明PyTorch框架与插件安装成功。

```
tensor([[[-0.6066,  6.3385,  0.0379,  3.3356],
         [ 2.9243,  3.3134, -1.5465,  0.1916],
         [-2.1807,  0.2008, -1.1431,  2.1523]], device='npu:0')
```

- 方法二

```
import torch
import torch_npu
```

```
x = torch.randn(2, 2).npu()
y = torch.randn(2, 2).npu()
z = x.mm(y)
```

```
print(z)
```

输出如下类似信息说明PyTorch框架与插件安装成功。

```
tensor([[[-0.0515,  0.3664],
         [-0.1258, -0.5425]], device='npu:0')
```

3.3 方式三：二进制软件包安装（abi1 版本）

本章节介绍如何获取并安装abi1版本的PyTorch框架和torch_npu插件。

安装 PyTorch 框架

使用以下命令可以直接从PyTorch官方获取abi1版本的安装包。

下载软件包

```
wget https://download.pytorch.org/whl/cpu-cxx11-abi/torch-<version>%2Bcpu.cxx11.abi-cp3x-cp3x-linux_x86_64.whl
```

安装命令

```
pip3 install torch-<version>+cpu.cxx11.abi-cp3x-cp3x-linux_x86_64.whl
```

示例命令如下：

```
wget https://download.pytorch.org/whl/cpu-cxx11-abi/torch-2.1.0%2Bcpu.cxx11.abi-cp39-cp39-linux_x86_64.whl
```

```
pip3 install torch-2.1.0+cpu.cxx11.abi-cp39-cp39-linux_x86_64.whl
```

说明

- 执行安装命令前，请参见[2 安装前准备](#)完成环境变量配置及相关依赖的安装。
- x表示8、9、10或11，即当前支持Python 3.8、Python 3.9、Python 3.10和Python 3.11。
- <version>表示PyTorch框架版本，当前指2.1.0、2.3.1、2.4.0或2.5.1。
- 当前暂不支持AArch64架构的abi1版本安装。

安装 torch_npu 插件

如下操作以PyTorch 2.1.0版本为例，介绍如何获取abi1版本的安装包并安装torch_npu插件。

1. 下载abi1版本安装包。

```
wget https://gitee.com/ascend/pytorch/releases/download/v7.0.0-pytorch2.1.0/torch_npu-2.1.0.post12_cxx11.abi_x86_64.zip
```

以v2.1.0-7.0.0为例，下载对应的Ascend Extension for PyTorch安装包，其他分支请参见《版本说明》中的“[相关产品版本配套说明](#)”章节。

2. 解压缩安装包。

```
unzip -o torch_npu-2.1.0.post12_cxx11.abi_x86_64.zip
```

3. 安装abi1版本torch_npu插件。

```
# 选择对应Python版本的安装包，如Python 3.9  
pip3 install torch_npu-2.1.0.post12+cxx11.abi-cp39-cp39-manylinux_2_28_x86_64.whl
```

 说明

- 执行安装命令前，请参见[2 安装前准备](#)完成环境变量配置及相关依赖的安装。
- PyTorch框架版本为2.1.0、2.3.1或者2.4.0，Python版本支持3.8~3.11；PyTorch框架版本为2.5.1，Python版本支持3.9~3.11。
- 当前暂不支持AArch64架构的abi1版本安装。

验证是否成功安装

• 方法一

```
python3 -c "import torch;import torch_npu; a = torch.randn(3, 4).npu(); print(a + a);"
```

输出如下类似信息说明PyTorch框架与插件安装成功。

```
tensor([[[-0.6066, 6.3385, 0.0379, 3.3356],  
        [ 2.9243, 3.3134, -1.5465, 0.1916],  
        [-2.1807, 0.2008, -1.1431, 2.1523]], device='npu:0')
```

• 方法二

```
import torch  
import torch_npu  
  
x = torch.randn(2, 2).npu()  
y = torch.randn(2, 2).npu()  
z = x.mm(y)  
  
print(z)
```

输出如下类似信息说明PyTorch框架与插件安装成功。

```
tensor([[[-0.0515, 0.3664],  
        [-0.1258, -0.5425]], device='npu:0')
```

4（可选）安装扩展模块

编译libtorch_npu

安装APEX模块

安装torchvision

4.1 编译 libtorch_npu

libtorch_npu是torch_npu插件的C++版本，包含运行torch_npu插件所需的头文件、库文件以及CMake配置文件。用户可以通过libtorch_npu使用torch_npu插件中开放的C++接口。

编译操作

步骤1 参见[2 安装前准备](#)、[3 安装PyTorch](#)完成依赖与PyTorch的安装。

步骤2 获取libtorch_npu源码。

```
git clone -b v2.1.0-7.0.0 https://gitee.com/ascend/pytorch.git
cd pytorch
git submodule update --init --recursive
```

以v2.1.0-7.0.0为例，拉取对应Ascend Extension for PyTorch分支代码。请参见《版本说明》中的“[相关产品版本配套说明](#)”章节下载Ascend Extension for PyTorch其他版本的分支代码。

步骤3 执行编译生成libtorch_npu安装包。

```
python3 build_libtorch_npu.py
```

编译时依赖的cmake需为3.18.0版本及以上，可参见[7.3 安装3.18.0版本cmake](#)。

默认编译release版本，如需debug版本，添加DEBUG=1环境变量。编译完成后，当前目录下生成libtorch_npu目录，包含以下文件。

- include：生成的C++头文件。
- lib：生成的C++库文件。
- share：包含Torch_npuConfig.cmake，用于用户编译构建时获取必要的头文件，库文件等配置文件。

----结束

libtorch 推理测试

以Ascend Extension for PyTorch源码仓v2.1.0-7.0.0分支下“pytorch/examples/libtorch_resnet”模型为例，介绍libtorch推理的快速使用。

步骤1 需提前安装torch、torch_npu、torchvision、hypothesis、expecttest以及packaging。

- torch与torch_npu、torchvision安装请参见[3 安装PyTorch](#)以及[4.3 安装torchvision](#)。
- hypothesis、expecttest、packaging安装可执行如下命令。如果使用非root用户安装，需要在命令后加--user，例如：**pip3 install expecttest --user**。
pip3 install expecttest
pip3 install packaging
pip3 install hypothesis

步骤2 编译文件添加NPU编译配置。

已完成NPU适配的编译文件请参见“pytorch/examples/libtorch_resnet/CMakeLists.txt”，可以直接用于编译工作。

用户如果使用自定义CMakeLists.txt编译文件，需添加以下内容用于引用libtorch_npu插件，以便进行后续基于NPU的编译工作。

```
set(torch_npu_path path_to_libtorch_npu) # 设置libtorch_npu的路径
include_directories(${torch_npu_path}/include) # 设置引用libtorch_npu的头文件路径
link_directories(${torch_npu_path}/lib) # 设置引用libtorch_npu的库文件路径

target_link_libraries(libtorch_resnet torch_npu) # 链接torch_npu库
```

步骤3 为了使模型在NPU设备上初始化和运行，用户需在C++代码中已将GPU接口修改为适配NPU的接口。当前脚本中已完成对应修改，用户可参见以下内容对实际开发的脚本进行修改。已完成NPU适配的模型代码文件请参见“pytorch/examples/libtorch_resnet/libtorch_resnet.cpp”。

代码示例如下，引入torch_npu头文件并设置初始化Device，在NPU使用结束时需要调用torch_npu::finalize_npu()释放资源，否则可能会有报错提示。

```
// 使用libtorch_npu相关接口，需引用libtorch_npu的头文件
#include<torch_npu/torch_npu.h>

// 使用NPU设备前需进行初始化
torch_npu::init_npu("npu:0");

// 通过传NPU字符串构造NPU设备
at::Device("npu:0")

// 使用NPU设备结束需进行反初始化
torch_npu::finalize_npu()
```

表 4-1 C++接口说明

接口	说明
torch_npu::init_npu()	使用NPU设备前需进行初始化，输入值格式为npu: <i>id</i> ，其中 <i>id</i> 为NPU卡号。
at::Device()	通过传NPU字符串构造NPU设备，输入值格式为npu: <i>id</i> ，其中 <i>id</i> 为NPU卡号。
torch_npu::finalize_npu() ()	使用NPU设备结束需进行反初始化，输入值格式为npu: <i>id</i> ，其中 <i>id</i> 为NPU卡号。

步骤4 执行编译并推理。

“pytorch/examples/libtorch_resnet/resnet_trace.py”脚本用于导出torchscript文件，可用于libtorch推理。

编译与推理脚本可参见“pytorch/ci/libtorch_resnet.sh”，当前提供的脚本已集成了导出torchscript文件、编译与推理部分，执行如下命令编译并推理：

```
bash libtorch_resnet.sh
```

显示以下内容，表示编译成功。

图 4-1 命令回显

```
-- Found Torch: /usr/local/python3.8.5/lib/python3.8/site-packages/torch/lib/libtorch.so
-- Found Torch_npu: /root/pytorch/libtorch_npu/lib/libtorch_npu.so
-- Configuring done
-- Generating done
-- Build files have been written to: /root/pytorch/examples/libtorch_resnet/build
[ 50%] Building CXX object CMakeFiles/libtorch_resnet.dir/libtorch_resnet.cpp.o
[100%] Linking CXX executable libtorch_resnet
[100%] Built target libtorch_resnet
Success.
[root@localhost ci]#
```

说明

aarch64环境下报torch.libs/*.so库不存在，请参见《常见问题》中的“[torch.libs/libopenblas-r0-56e95da7.3.24.so不存在](#)”章节。

----结束

4.2 安装 APEX 模块

APEX混合精度模块是一个集优化性能、精度收敛于一身的综合优化库，可以提供不同场景下的混合精度训练支持。

混合精度训练是在训练时混合使用单精度（float32）与半精度（float16）数据类型，将两者结合在一起，并使用相同的超参数实现了与float32几乎相同的精度。在迁移完成、训练开始之前，基于NPU硬件的架构特性，用户可以选择开启混合精度，用以提升模型的性能。

推荐用户通过编译源码包安装APEX模块。

方式一（推荐）：容器场景

1. 下载APEX源码。

```
git clone -b master https://gitee.com/ascend/apex.git
```

2. 构建镜像。

```
cd apex/scripts/docker/{arch}
docker build -t manylinux-builder:v1 .
```

说明

- {arch}表示CPU架构（X86或ARM）。
 - 注意不要遗漏命令结尾的“.”。
3. 进入Docker容器，并将APEX源代码挂载至容器内。

```
docker run -it -v {{code_path}}/apex:/home/apex manylinux-builder:v1 bash
```

{code_path}表示APEX源代码路径，请根据实际情况进行替换。
 4. 安装torch。

```
pip3.8 install torch==2.1.0
```

以Python3.8、PyTorch 2.1.0为例，其他Python及torch版本请根据实际场景修改命令再执行。

5. 编译生成二进制安装包。

```
cd /home/apex
bash scripts/build.sh --python=3.8
```

如需指定其他Python版本请使用--python=3.9、--python=3.10或--python=3.11。

命令完成后进入apex/dist/目录，可获取wheel格式的二进制安装包。

📖 说明

如果返回类似如下回显信息：

```
fatal: unable to access 'https://gitee.com/ascend/samples.git/': Peer's certificate issuer has been marked as not trusted by the user.
```

则执行git config --global http.sslVerify "false"命令关闭证书检验，再执行编译命令。

6. 在运行环境中如下命令进行安装。如果使用非root用户安装，需要在命令后加--user。

```
pip3 uninstall apex # 可选，若当前未安装apex，可不执行
pip3 install apex-0.1+ascend-cp3x-cp3x-arch.whl # x指Python版本尾号，arch指CPU架构
```

方式二：物理机及虚拟机场景

1. 安装依赖。

选择编译安装方式安装时需要安装系统依赖，根据不同类型的操作系统，选择对应的命令安装所需依赖。

- openEuler系列（openEuler、CentOS、Kylin、BCLinux、BC-Linux-for-Euler、UOS20 1050e、UOS20 1020e、UOSV20、AntOS、CTyunOS、CULinux、Tlinux）：

- i. 安装依赖（除gcc和cmake以外）。

```
yum install -y patch libjpeg-turbo-devel dos2unix openblas git
```

- ii. 安装gcc和cmake。

根据实际情况，安装对应gcc和cmake版本，版本信息及安装指导请参见以下说明。

- Debian系列（Ubuntu、Debian、UOS20、UOS20 SP1）：

- i. 安装依赖（除gcc和cmake以外）。

```
apt-get install -y patch build-essential libbz2-dev libreadline-dev wget curl llvm
libncurses5-dev libncursesw5-dev xz-utils tk-dev liblzma-dev m4 dos2unix libopenblas-dev
git
```

- ii. 安装gcc和cmake。

根据实际情况，安装对应gcc和cmake版本，版本信息及安装指导请参见以下说明。

📖 说明

AArch64架构安装gcc10.2.0版本，X86_64架构安装gcc9.4.0版本，cmake则需为3.18.0版本及以上，安装指导可参见[7.2 安装10.2.0版本gcc](#)和[7.3 安装3.18.0版本cmake](#)。

2. 请确保已安装PyTorch框架且setuptools版本小于等于65.7.0，若版本不符合条件，可使用以下命令安装。

```
pip3 install setuptools==65.7.0
```

3. 获取昇腾适配的APEX源码。

```
git clone -b master https://gitee.com/ascend/apex.git
```

4. 进入昇腾适配的APEX源码目录，执行命令编译生成二进制安装包。
- ```
cd apex
bash scripts/build.sh --python=3.8
```

 **说明**

支持3.8、3.9、3.10及3.11，并确保NPU版本的PyTorch可以正常使用，否则会影响APEX的编译。

命令执行过程中会自动拉取APEX官方源码，请保证网络畅通，完成后将在apex/dist目录下生成二进制安装包。

5. 执行如下命令进行安装。如果使用非root用户安装，需要在命令后加--user。
- ```
cd apex/dist/
pip3 uninstall apex # 可选，若当前未安装apex，可不执行
pip3 install apex-0.1+ascend-cp3x-cp3x-arch.whl # x指Python版本尾号，arch指CPU架构
```
- 命令示例：
- ```
若用户在x86架构下安装，请将命令中文件包名中的“aarch64”改为“x86_64”。
pip3 install apex-0.1+ascend-cp38-cp38-linux_aarch64.whl
```

 **说明**

APEX模块的使用介绍可单击[Link](#)进行参考。

## 4.3 安装 torchvision

torchvision为PyTorch的扩展库，主要用于计算机视觉任务。它包括许多常用的模型、数据集，使得处理图像数据变得更加简单高效。

安装PyTorch框架和torch\_npu插件后，需安装对应框架版本的torchvision，具体请参见下表。

表 4-2 torchvision 版本配套

| PyTorch版本 | torchvision版本 |
|-----------|---------------|
| 2.1.0     | 0.16.0        |
| 2.3.1     | 0.18.1        |
| 2.4.0     | 0.19.0        |
| 2.5.1     | 0.20.1        |

执行如下命令安装torchvision，以0.16.0版本为例：

```
pip3 install torchvision==0.16.0
```

# 5 卸载

## 卸载 Ascend Extension for PyTorch

- 卸载PyTorch框架需执行如下命令：  
`pip3 uninstall torch`
- 卸载torch\_npu插件需执行如下命令：  
`pip3 uninstall torch_npu`
- 卸载APEX模块需执行如下命令：  
`pip3 uninstall apex`

### 说明

如需要保存卸载日志，可在pip3 uninstall命令后面加上参数--log <path>，并对您指定的目录<path>做好权限管控。

## 卸载 CANN 软件

如果用户只卸载CANN软件包（如ToolKit），则卸载没有先后顺序，如果还要卸载驱动和固件，则需要卸载其他软件包以后再卸载驱动和固件。操作步骤如下：

- 步骤1** 进入软件的卸载脚本所在路径，一般放置在“script”目录下（“script”所在路径请以实际为准）。

```
cd <path>/ascend-toolkit/<version>/<arch>-linux/script
```

其中<path>为软件包的安装路径，<version>为软件包版本，<arch>-linux为CPU架构，请用户根据实际情况替换。

- 步骤2** 执行./uninstall.sh命令运行脚本，完成卸载。

卸载完成后，若显示如下信息，则说明软件卸载成功：

```
[INFO] xxx uninstall success
```

xxx表示卸载的实际软件包名。

----结束

## 卸载驱动固件

驱动和固件的卸载没有先后顺序要求，操作步骤如下：

- 步骤1** 使用PuTTY登录服务器的OS命令行。

**步骤2** 执行如下命令，切换至root用户。

```
su - root
```

**步骤3** 在任意路径执行如下命令卸载软件包。

- 卸载固件  
`<install_path>/firmware/script/uninstall.sh`
- 卸载驱动  
`<install_path>/driver/script/uninstall.sh`

 **说明**

`<install_path>`表示软件包安装路径，可以执行`cat /etc/ascend_install.info`命令查询安装路径，请根据实际情况替换。

**步骤4 可选:** 根据系统提示信息决定是否重启服务器，若需要重启系统，请执行以下命令；否则，请跳过此步骤。

```
reboot
```

----**结束**

# 6 升级

---

**步骤1** 需要先手动卸载旧版本，具体可参见[5 卸载](#)。

**步骤2** 安装新版本，具体可参见[3 安装PyTorch](#)进行操作。

----结束

## 说明

- 在驱动、固件和CANN软件配套场景下，可单独升级PyTorch框架与torch\_npu插件。
- 如果也要升级固件、驱动和CANN软件，请按照“固件->驱动->CANN软件”的顺序先进行升级，再升级PyTorch框架与torch\_npu插件。

# 7 参考信息

配置pip源

安装10.2.0版本gcc

安装3.18.0版本cmake

## 7.1 配置 pip 源

配置pip源，配置方法如下：

**步骤1** 使用软件包的安装用户，执行如下命令：

```
cd ~/.pip
```

如果提示目录不存在，则执行如下命令创建：

```
mkdir ~/.pip
cd ~/.pip
```

**步骤2** 编辑pip.conf文件。

使用**vi pip.conf**命令打开pip.conf文件，写入如下内容：

```
[global]
#以华为源为例，请根据实际情况进行替换。
index-url = https://mirrors.huaweicloud.com/repository/pypi/simple
trusted-host = mirrors.huaweicloud.com
timeout = 120
```

**步骤3** 执行**wq!**命令保存文件。

----结束

## 7.2 安装 10.2.0 版本 gcc

以root安装gcc 10.2.0为例演示编译安装操作，编译前请检查系统时间，确认与当前一致后再进行编译，避免编译安装gcc时陷入死循环。

**步骤1** 在服务器任意目录（如“/home”）下，执行如下命令获取gcc-10.2.0.tar.gz源码包。

```
wget https://repo.huaweicloud.com/gnu/gcc/gcc-10.2.0/gcc-10.2.0.tar.gz
```

若该命令报错，用户可单击[Link](#)手动下载并上传源码包。

**步骤2** 安装gcc的时候会占用大量临时空间，所以先执行下面的命令清空/tmp目录：

```
rm -rf /tmp/*
```

**步骤3** 安装依赖（以Debian系列和openEuler系列操作系统为例）。

- openEuler系列执行如下命令安装。  

```
yum install bzip2
```
- Debian系列执行如下命令安装。  

```
apt-get install bzip2
```

**步骤4** 编译安装gcc。

1. 进入gcc-10.2.0.tar.gz源码包所在目录，解压源码包，命令为：  

```
tar -zxvf gcc-10.2.0.tar.gz
```

2. 进入解压后的文件夹，执行如下命令下载gcc依赖包：  

```
cd gcc-10.2.0
./contrib/download_prerequisites
```

执行上述命令若显示类似如下报错：

```
gmp-6.1.0.tar.bz2: FAILED
sha512sum: WARNING: 1 computed checksum did NOT match
error: Cannot verify integrity of possibly corrupted file gmp-6.1.0.tar.bz2
```

表示可能因网络原因gmp包没有下载完全，可执行**rm -rf gmp-6.1.0.tar.bz2**命令删除gmp包，再执行如下命令在“gcc-10.2.0/”文件夹下手动下载依赖包：

```
wget http://gcc.gnu.org/pub/gcc/infrastructure/gmp-6.1.0.tar.bz2
wget http://gcc.gnu.org/pub/gcc/infrastructure/mpfr-3.1.4.tar.bz2
wget http://gcc.gnu.org/pub/gcc/infrastructure/mpc-1.0.3.tar.gz
wget http://gcc.gnu.org/pub/gcc/infrastructure/isl-0.18.tar.bz2
```

下载好上述依赖包后，重新执行以下命令：

```
./contrib/download_prerequisites
```

3. 执行配置、编译和安装命令：  

```
./configure --enable-languages=c,c++ --disable-multilib --with-system-zlib --prefix=/usr/local/gcc10.2.0
make -j15 # 通过grep -w processor /proc/cpuinfo|wc -l查看cpu数，示例为15，用户可自行设置相应参数。
make install
```

#### 说明

- 编译耗时1小时左右，请用户耐心等待。
- 其中“--prefix”参数用于指定gcc10.2.0安装路径，用户可自行配置，但注意不要配置为“/usr/local”及“/usr”，因为会与系统使用软件源默认安装的gcc相冲突，导致系统原始gcc编译环境被破坏。示例指定为“/usr/local/gcc10.2.0”。

**步骤5** 配置环境变量（请在实际需要时再进行配置）。

例如用户在启动在线推理或训练进程前需执行如下命令配置环境变量。

```
export LD_LIBRARY_PATH=/usr/local/gcc10.2.0/lib64:${LD_LIBRARY_PATH}
export CC=/usr/local/gcc10.2.0/bin/gcc
export CXX=/usr/local/gcc10.2.0/bin/g++
export PATH=/usr/local/gcc10.2.0/bin:${PATH}
```

其中“/usr/local/gcc10.2.0”为**步骤4.3**中配置的gcc10.2.0安装路径，请根据实际情况替换。

**步骤6** 执行**gcc --version**命令查看gcc版本，若返回如下版本信息，表示安装成功。

```
gcc (GCC) 10.2.0
```

----结束

**须知**

如果用户明确需要gcc 10.2.0编译，且由于用户编译脚本等问题，无法通过环境变量控制gcc版本时，可执行如下操作，修改软链接。

**1. 备份旧版本软链接。**

```
mv /usr/bin/gcc /usr/bin/gcc.bak
mv /usr/bin/g++ /usr/bin/g++.bak
mv /usr/bin/c++ /usr/bin/c++.bak
mv /usr/bin/cc /usr/bin/cc.bak
```

**2. 建立新版本软链接。**

```
ln -s /usr/local/gcc10.2.0/bin/gcc /usr/bin/gcc
ln -s /usr/local/gcc10.2.0/bin/g++ /usr/bin/g++
ln -s /usr/local/gcc10.2.0/bin/c++ /usr/bin/c++
ln -s /usr/local/gcc10.2.0/bin/cc /usr/bin/cc
```

**3. 执行gcc --version命令查看gcc版本，若返回如下版本信息，表示配置成功。**

```
gcc (GCC) 10.2.0
```

## 7.3 安装 3.18.0 版本 cmake

以下步骤请在root用户下执行。

**步骤1** 在服务器任意目录下（如“/home”），执行如下命令获取cmake软件包。

```
wget https://cmake.org/files/v3.18/cmake-3.18.0.tar.gz
```

**步骤2** 解压并进入软件包目录。

```
tar -xvf cmake-3.18.0.tar.gz
cd cmake-3.18.0/
```

**步骤3** 执行配置、编译和安装命令。

```
./configure --prefix=/usr/local/cmake
make && make install
```

**步骤4** 设置软连接。

```
ln -s /usr/local/cmake/bin/cmake /usr/bin/cmake
```

如果执行上述命令报“ln: failed to create symbolic link '/usr/bin/cmake': File exists”错误，显示已有软连接（'/usr/bin/cmake'），则执行如下命令删除软连接（软连接'/usr/bin/cmake'仅为示例，请用户以实际情况为准）：

```
rm -rf /usr/bin/cmake
```

再执行ln -s /usr/local/cmake/bin/cmake /usr/bin/cmake命令设置软连接。

**步骤5** 执行如下命令验证是否安装成功。

```
cmake --version
```

如显示“cmake version 3.18.0”则表示安装成功。

----结束